

Assembly Practice Problems

1. In this problem, you will write a *complete assembly program* (i.e., an assembly program along with any relevant assembler directives) to manipulate two data tables, one being an input table located within program memory and one being an output table located within data memory. Throughout the problem, assume that the symbols *IN_TAB* (representing address 0xD007) and *OUT_TAB* (representing address 0x2A0E) must be used to specify the starting addresses for the input table and output table, respectively. Additionally, assume that the *input table* will, at most, consist of 0x4AB *pairs of 8-bit numbers*, with the end of the input table being denoted by both bytes within a pair being equal.

Design a complete assembly program that, *for each successive pair of 8-bit values within the relevant input table* (i.e., do the following to all 8-bit pairs within the input table), (1) loads an 8-bit pair from the input table, (2) stores each byte within the pair to the next available locations within the output table *in reverse order* (i.e., second byte first, first byte second), (3) calculates the sum of the pair of bytes, (4) clears the most-significant bit and sets the least significant bit within the resulting sum, and (5) stores this altered sum to the next available location within the output table. Again, assume that the end of the input table will be denoted by both bytes within a pair being equal. Define the relevant input table to consist of at least three pairs of elements, allocate an appropriate size for the output table (following the assumption that there could be at most 0x4AB pairs of 8-bit numbers within the input table), and have a main routine that, after any relevant initializations, performs the aforementioned operations for the entire input table before entering an infinite loop.

[illegible]

Assembly Practice Problems

[illegible]

Assembly Practice Problems

2. Assuming that the values within R16 and R17 are unsigned and only use the lower four bits of the register (i.e., that they are essentially unsigned 4-bit values), write an assembly program to multiply the value within register R16 by the value within register R17 without using instructions that perform multiplication or shifting. (Recall that for all whole numbers x and y , $x \times y = \sum_1^y x$; for example, $4 \times 3 = 4 + 4 + 4$.) Store the final result in R19.

[illegible]

Assembly Practice Problems

3. Assuming that there is a table of **237** 16-bit signed words starting at program memory location **0x8100**, write a program fragment to configure the appropriate index and determine how many of the **words** represent **negative** values. At the end of your fragment, the result should be stored in R16.

[illegible]

Assembly Practice Problems

4. Below, write an **efficient program fragment** (i.e., one with a minimum number of instructions) to jump to the program memory address specified by the label ***LOL_37*** if the value within register R16 (value unsigned) is greater than or equal to unsigned decimal 37. Use the ***minimum*** number of instructions.

Assembly Code	Comments

5. Use the **simplest** (most efficient) assembly language to store the value 0x37 to 100 consecutive address locations in data memory, starting at the address determined by *X*. Assume the pointer *X* has already been initialized to a particular starting address in data memory. Write this code as a program fragment

Assembly Code	Comments

6. Write a program fragment that computes the average between two unsigned values in registers *R16* and *R17* and then stores the results directly to data memory, i.e. $Avg = (R16 + R17) / 2$. If *Avg* is greater than or equal to 100, store *Avg* to address 0x3744 in data memory. Otherwise, store *Avg* to address 0x3701 in data memory.

Assembly Code	Comments